Campus guide – local spot review

INTRODUCTION:

Campus Guide System is a web-based application developed to help students easily find useful places around their college campus. New students often face difficulty in locating good food places, stationery shops, PG accommodations, and other essential services. Most of the time, they depend on friends or informal sources, which may not always be reliable.

This system provides a centralized platform where students can explore nearby places, read reviews, and share their own experiences. The goal of this project is to make campus life more convenient by providing trusted information in one place.

SCENARIO BASED CASE STUDY:

Consider a student named Riya who has just joined college. She is new to the campus and does not know where to find good food or affordable services.

Riya opens the Campus Guide application and registers using her details. After logging in, she browses different categories like food, stationery, and accommodation. She checks ratings and reviews given by other students.

She finds a highly rated canteen near her campus and decides to visit it. After her visit, she comes back and

writes her own review, helping other students make better decisions.

At the same time, a shop owner can claim their business on the platform and update information such as contact details and timings. This creates a helpful system for both students and local businesses.

SYSTEM REQUIREMENTS:

These are the essential tools and platforms required to develop, test, and run the application efficiently.

Software Requirements

These are the tools and technologies required to develop, run, and test the Campus Guide System effectively.

Operating System: Windows 10/11, macOS, or Linux — Supports development and testing across different platforms.

Node.js (v16 or above): Used as the runtime environment to handle backend operations and API requests.

npm (v8 or above): Package manager used to install and manage dependencies required for the project.

React.js: A JavaScript library used to build an interactive and user-friendly frontend interface.

Browser: Google Chrome or Mozilla Firefox (latest version) Used to run and test the application in real time.

Express.js: A backend framework used to create APIs and handle server-side logic.

MongoDB: A NoSQL database used to store user data, business listings, and reviews in a flexible format.
Postman: Tool used to test API endpoints during development.

Visual Studio Code: Code editor used for writing and managing the project with built-in terminal and Git support.

Hardware Requirements

These are the minimum and recommended system specifications needed to develop and run the application smoothly.

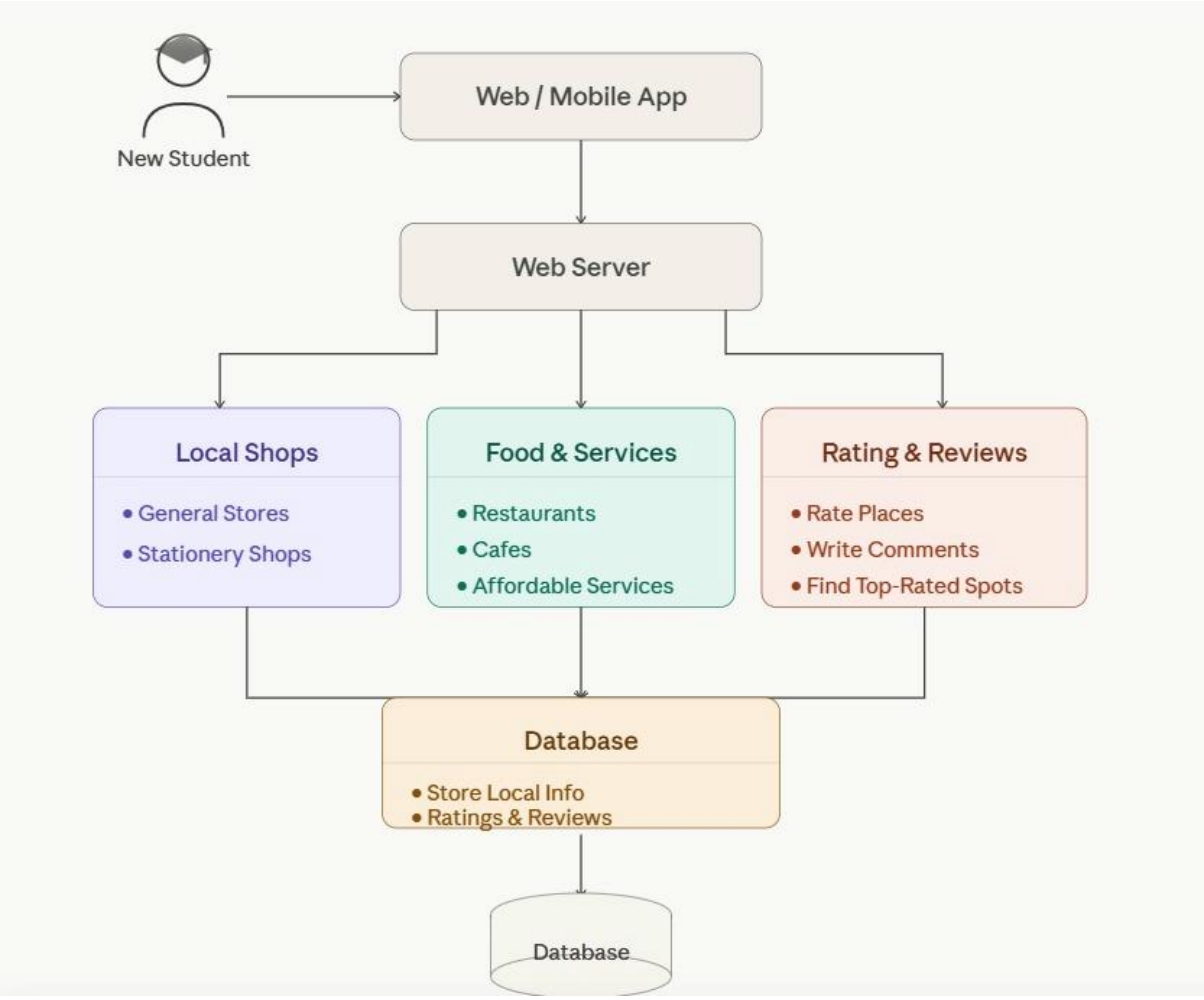
Processor: Intel Core i5 (8th Gen or above) or AMD Ryzen 5 — Required for smooth coding and application performance.

RAM: Minimum 8 GB (16 GB recommended) — Helps in running development tools, servers, and browser simultaneously.

Storage: At least 1 GB free space — Required for installing dependencies and storing project files.

Display: 1366×768 resolution or higher — Ensures better visibility and layout during development.

PROJECT ARCHITECTURE:



The Campus Guide System follows a structured architecture that shows how users interact with the application and how data flows through the system.

The diagram above represents the overall working of the system.

A new student accesses the application through a web or mobile interface. The request is then sent to the web server, which processes the user's actions and directs them to different modules of the system.

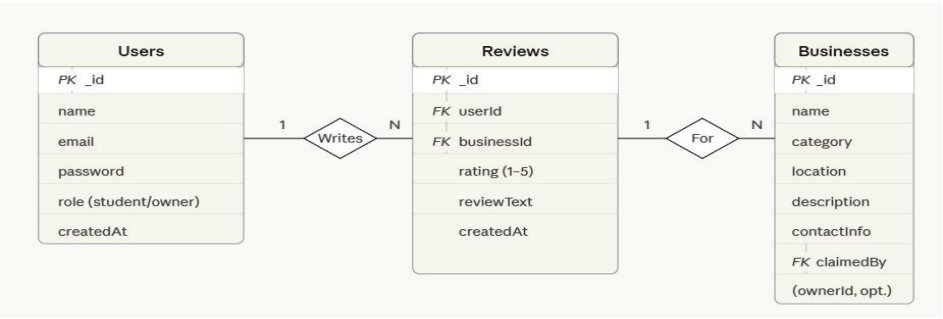
The application is divided into three main functional sections:

- **Local Shops:**
  This module helps students find nearby stores such as general stores and stationery shops.
- **Food & Services:**
  This section includes restaurants, cafes, and other affordable services available near the campus.
- **Rating & Reviews:**
  Students can rate places, write reviews, and explore top-rated locations based on user feedback.

All the information related to businesses, user reviews, and ratings is stored in the database. The database ensures that data is organized and easily accessible whenever required.

This architecture helps in maintaining a smooth flow of information between the user interface, server, and database, making the system efficient and user-friendly.

ER DIAGRAM:



The Entity-Relationship (ER) diagram of the Campus Guide System represents the structure of the database and the relationships between different entities in the system. It helps in understanding how data is organized and connected.

1. Entities

The main entities in the system are:

- **Users:**
This entity stores information about the users of the system.
Attributes include user ID, name, email, password, role (student or owner), and account creation date.

- **Businesses:**
This entity contains details of local campus spots such as shops, restaurants, and services.
Attributes include business ID, name, category, location, description, contact information, and claim status.

- **Reviews:**
This entity stores the feedback given by users for different businesses.
Attributes include review ID, user ID, business ID, rating (1 to 5), review text, and creation date.

2. Relationships

The relationships between entities are as follows:

User – Review (Writes):
A user can write multiple reviews, but each review is written by only one user.
This represents a one-to-many relationship.

Business – Review (Receives):
A business can have multiple reviews, but each review belongs to one business.
This is also a one-to-many relationship.

3. Primary Keys

Each entity contains a primary key to uniquely identify its records:

- User → _id-
- Business → _id-
- Review → _id

4. Foreign Keys

Foreign keys are used to establish relationships between entities:

- userId in Reviews refers to Users
- businessId in Reviews refers to Businesses

The ER diagram clearly shows how users interact with businesses through reviews. Users can share their experiences by rating and reviewing different campus spots, while businesses can collect feedback from multiple users.

This structure ensures proper data organization and helps in efficient data retrieval for the application.

KEY FEATURES

The Campus Guide System provides several useful features that help students easily explore and review places around their campus. The main features of the system are described below:

User Registration and Authentication

The system allows users to create an account and log in securely. This ensures that only registered users can add reviews and access personalized features.

Business Directory

Users can browse a list of nearby campus spots such as food places, stationery shops, and other services. The listings are organized into categories for easy navigation.

Search and Category Filtering

Students can filter places based on categories like restaurants, cafes, or shops. This helps users quickly find what they are looking for without wasting time.

Review and Rating System

Users can rate places on a scale of 1 to 5 and write their experiences. This feature helps other students make informed decisions based on real feedback.

Top-Rated Section

The system highlights the best places based on user ratings and reviews. This allows users to easily discover popular and trusted spots.

Business Claim Feature

Shop owners can claim their business listing and update details such as contact information, timings, and descriptions. This keeps the information accurate and up to date.

User-Friendly Interface

The application is designed with a simple and clean interface, making it easy for users to navigate and use all features without confusion.

Secure Data Management

User data, business details, and reviews are stored securely in the database, ensuring privacy and reliability of information.

ROLES AND RESPONSIBILITIES

The Campus Guide System involves different types of users, each having specific roles and responsibilities to ensure the smooth functioning of the platform.

User (Student):

- **Registration and Account Management:**
  Users are responsible for creating an account by providing accurate details and maintaining their login credentials securely.
- **Browsing and Exploration:**
  Users can explore different categories such as food, shops, and services to find useful places around the campus.
- **Review and Rating:**
  Users can rate places and write reviews based on their personal experience. They should provide honest and helpful feedback.
- **Decision Making:**
  Users can read reviews and ratings given by others to choose the best places according to their needs.

Business Owner:

- **Business Registration / Claiming:**
  Owners can claim their business listing on the platform and verify their ownership.
- **Managing Business Information:**
  They can update details such as business name,

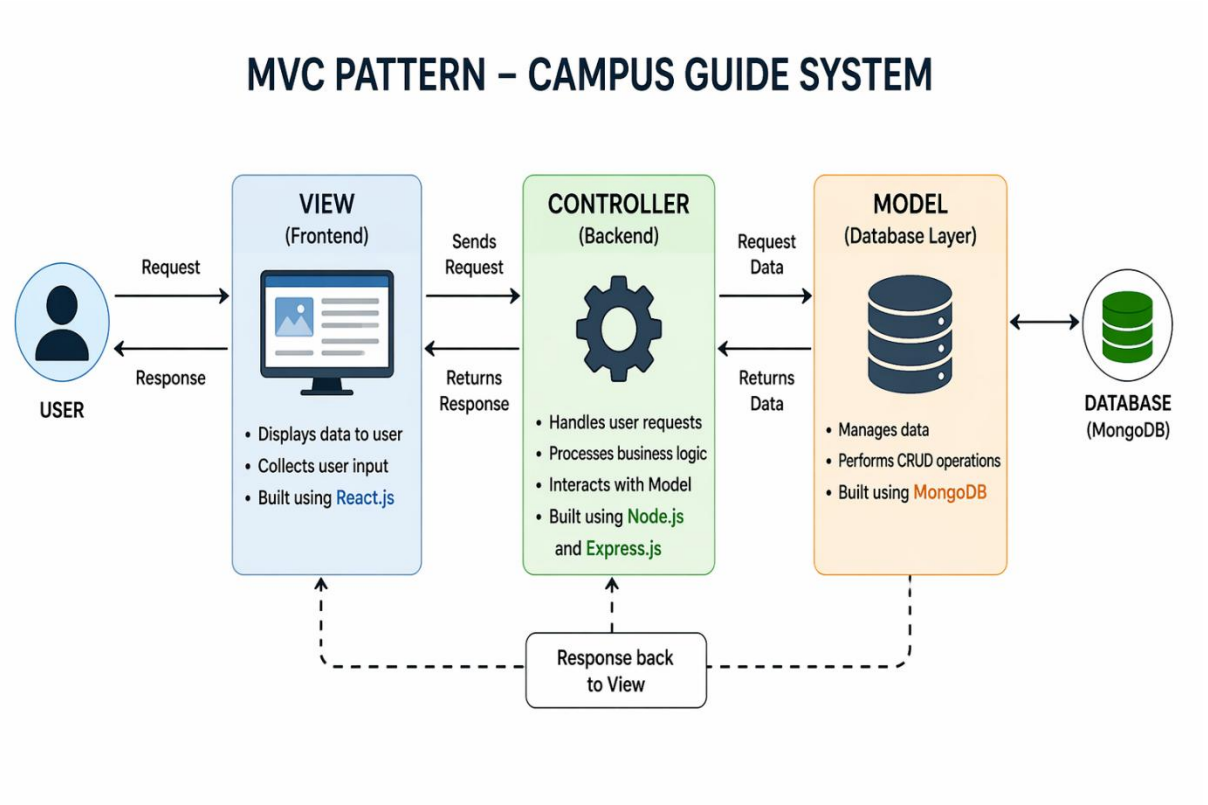
contact information, location, and timings to keep the information accurate.

- **Monitoring Reviews:**
  Owners can view customer reviews and ratings to understand user feedback and improve their services.

Admin:

- **System Management:**
  The admin is responsible for managing the overall system, ensuring that it runs smoothly without errors.
- **User Management:**
  Admin monitors user accounts and can take action in case of misuse or inappropriate behavior.
- **Business Listing Management:**
  Admin can add, update, or remove business listings to maintain quality and accuracy.
- **Content Moderation:**
  Admin ensures that reviews and content posted by users follow proper guidelines and are appropriate

MVC PATTERN



The Campus Guide System follows the **Model-View-Controller (MVC)** architecture. This design pattern helps in organizing the application into separate layers, making it easier to develop, manage, and update.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related operations. It defines the structure of the database and manages how data is stored, updated, and retrieved.

In this project, the Model includes collections such as Users, Businesses, and Reviews, which are managed using MongoDB and Mongoose.

View Layer (Frontend Layer)

The View layer represents the user interface of the application. It is responsible for displaying data to the user and collecting user input.

In the Campus Guide System, the View is developed using React.js and includes pages like login, home, business listings, and review forms.

Controller Layer (Logic Layer)

The Controller acts as a bridge between the Model and the View. It processes user requests, performs necessary operations using the Model, and sends the response back to the View.

For example, when a user submits a review, the controller receives the request, stores the data in the database, and sends a response back to the frontend.

Advantages of MVC Pattern

- Clear separation of components
- Easy to maintain and update
- Improves code reusability
- Supports teamwork and scalability

PROJECT SETUP AND CONFIGURATION:

To develop and run the Campus Guide System successfully, the project needs to be properly set up. The setup is divided into two main parts: Client-side (Frontend) and Server-side (Backend).

Creating Project Structure

1. Create a main project folder with the name *Campus Guide System*.
2. Inside the folder, create two subfolders:
    - **Client** (for frontend)
    - **Server** (for backend)
3. Open the project folder in Visual Studio Code.

Client Setup (Frontend Setup)

The frontend is developed using React.js.

● Open the **Client** folder in the terminal.

● Create a React application using Vite:

npm create vite@latest . -- --template react

● Select React framework and JavaScript variant.

● Install all required dependencies:

npm install

● Start the development server:

npm run dev

This will run the frontend application in the browser.

Server Setup (Backend Setup)

The backend is developed using Node.js and Express.js.

● Open the **Server** folder in the terminal.

● Initialize a Node.js project:

npm init -y

● Install required packages:

npm install express mongoose cors dotenv bcrypt jsonwebtoken

● Create the following files and folders:

- **server.js** (main server file)
- **models/** (for database schemas)
- **controllers/** (for handling logic)
- **routes/** (for API endpoints)
- **middleware/** (for authentication)

Database Configuration

● Install MongoDB and connect it to the project.

● Use Mongoose to create schemas and models.

● Ensure the database connection is established before running the server.

Running the Application

● Start the backend server:

node server.js

● Start the frontend server:

npm run dev

Now, the application will run successfully, and users can access the Campus Guide System through the browser.

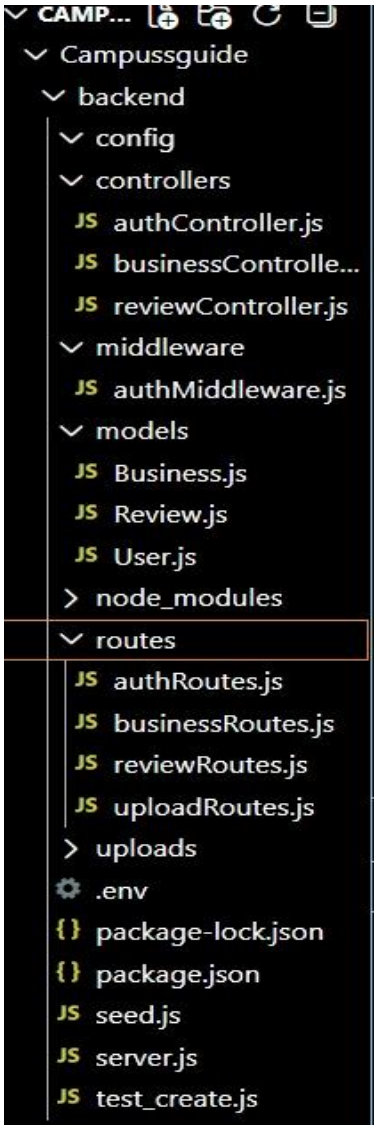
This setup ensures that both frontend and backend are properly connected and the system runs smoothly. A clear project structure also makes development easier and more organized.

Backend Development :

Backend reference video :

https://drive.google.com/file/d/1wVv3ihVCaSUOB3HunMDnwEHu3C25WQTV/view?usp=sharing

Backend structure :



Controllers

- **authController.js →** Handles all authentication-related operations (user registration, login, token generation, etc.).

- **businessController.js →** Handles all business-related operations (add businesses, view listings, update details, delete listings, etc.).

- **reviewController.js →** Handles all review and rating operations (add review, view reviews, update/delete reviews, etc.).

db

- **config.js →** Contains database connection setup using MongoDB and Mongoose.

middleware

- **authMiddleware.js →** Middleware for authentication and authorization (JWT validation and protected routes).

Models

- User

User.js → Schema/model for storing user details such as name, email, password, and role.

- **Business**

Business.js → Schema/model for storing business details like name, category, location, and contact information.

Review

Review.js → Schema/model for storing user reviews, ratings, and feedback for businesses.

routes

- **authRoutes.js →** Defines API routes for authentication actions (register, login, etc.).

- **businessRoutes.js →** Defines API routes for business-related operations (add, view, update, delete businesses).

- **reviewRoutes.js →** Defines API routes for review-related operations (add review, fetch reviews, etc.).

- **uploadRoutes.js →** Defines API routes for handling file uploads (images or other media if used).

DATABASE DEVELOPMENT

1. Configure MongoDB

To store and manage data efficiently, MongoDB is used as the database for the Campus Guide System.

- Install Mongoose:
Mongoose is used to interact with MongoDB and define schemas for the application.

In your Node.js project directory, run:

npm install mongoose

Create Database Connection

Before storing or retrieving data, the database must be connected.
The connection is made in the backend using Mongoose with the MongoDB URL.

Campussguide > backend > config > JS db.js > ...

```javascript
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas

Schemas are created to define how data will be stored in the database. Based on the project design, the main schemas used in the Campus Guide System are:

1) User.js → Schema/model for user accounts

Stores user details such as:Name , Email,Password, Role (student/owner) , Created date

Campussguide > backend > models > User.js > userSchema > password

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
    },
    password: {
      type: String,
      required: true,
    },
    role: {
      type: String,
      enum: ['student', 'owner'],
      default: 'student',
    },
```

Campussguide > backend > models > User.js > userSchema > password

```
const userSchema = mongoose.Schema(
  },
  {
    timestamps: true,
  }
);

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

const User = mongoose.model('User', userSchema);
module.exports = User;
```

2) Business.js → Schema/model for campus businesses

Stores details of local campus spots such as:

Business name, Category (food, shops, services),

Location, Description, Contact information, Claimed status

Campussguide > backend > models > JS Business.js > ...

```
1  const mongoose = require('mongoose');
2
3  const businessSchema = mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true,
8      },
9      category: {
10       type: String,
11       required: true,
12     },
13     location: {
14       type: String,
15       required: true,
16     },
17     lat: {
18       type: Number,
19     },
20     lng: {
21       type: Number,
22     },
23     description: {
24       type: String
```

Campussguide > backend > models > JS Business.js > ...

```
3  const businessSchema = mongoose.Schema(
23     description: {
24       type: String,
25       required: true,
26     },
27     contactInfo: {
28       type: String,
29     },
30     claimedBy: {
31       type: mongoose.Schema.Types.ObjectId,
32       ref: 'User',
33     },
34     imageUrl: {
35       type: String,
36     },
37     averageRating: {
38       type: Number,
39       default: 0,
40     },
41   },
42   {
43     timestamps: true,
44   }
```

```
Campussguide > backend > models > JS Business.js > ...
const businessSchema = mongoose.Schema(
    },
    averageRating: {
      type: Number,
      default: 0,
    },
  },
  {
    timestamps: true,
  }
);

const Business = mongoose.model('Business', businessSchema);
module.exports = Business;
```

3) Review.js → Schema/model for user reviews

Stores feedback given by users:

User ID , Business ID , Rating (1–5) , Review text , Created date

```
Campussguide > backend > models > JS Review.js > ...
const mongoose = require('mongoose');

const reviewSchema = mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    business: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Business',
      required: true,
    },
    rating: {
      type: Number,
      required: true,
      min: 1,
      max: 5,
    },
    reviewText: {
      type: String,
      required: true,
```

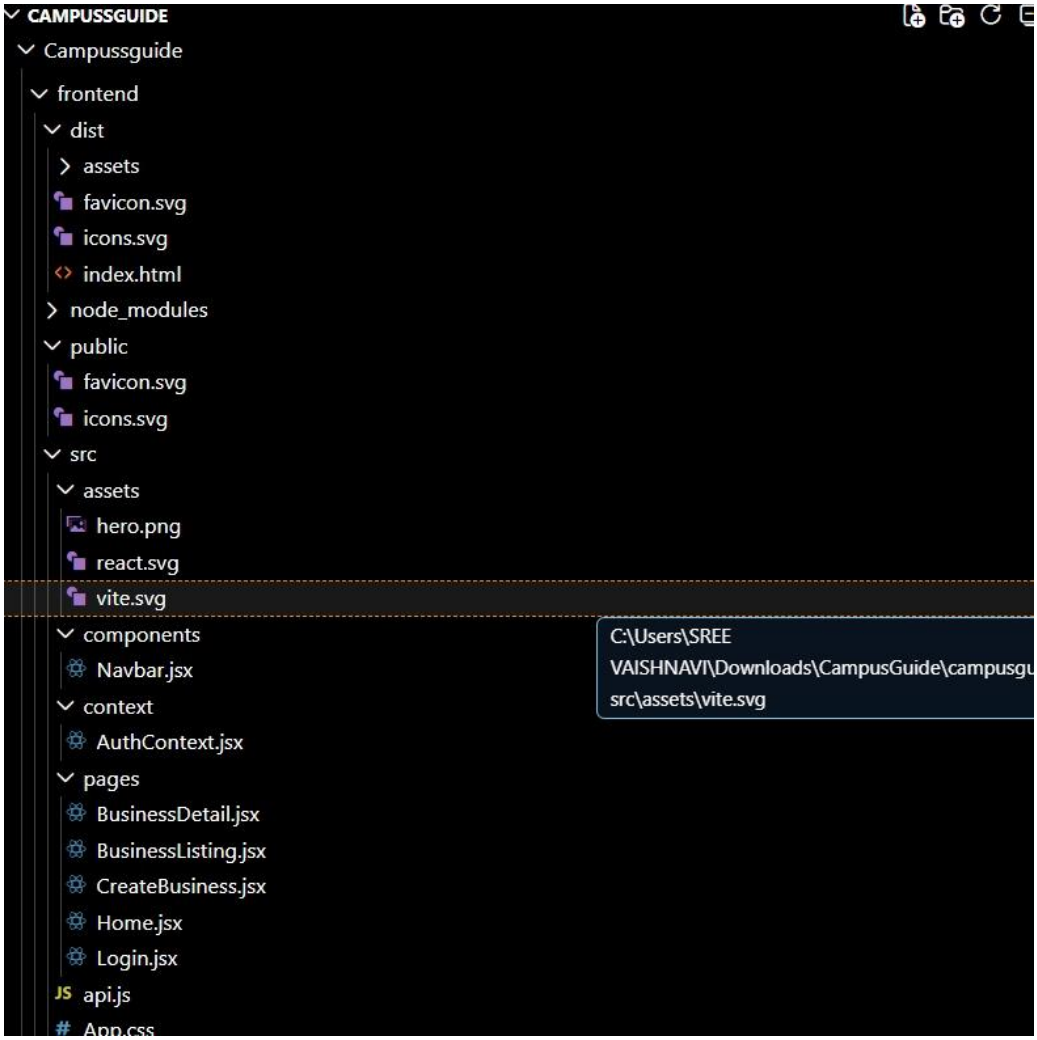
```
        required: true,
      },
    },
    {
      timestamps: true,
    }
  );

  const Review = mongoose.model('Review', reviewSchema);
  module.exports = Review;
```

FRONT-END DEVELOPMENT:

Frontend Reference Video:

https://drive.google.com/file/d/1Gkv6tLENLLpOAUxuPmlNBPsL9zymf47W/view?usp=drive_link

Frontend Structure:



```
App.css
App.jsx
index.css
main.jsx
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
vite.config.js
README.md
```

- **Home.jsx** → Home page of the application.

- **Login.jsx** → User login page.

- **BusinessListing.jsx** → Displays list of all businesses**.**

- **BusinessDetail.jsx** → Shows details of a selected business.

- **CreateBusiness.jsx** → Page to add a new business.

Components

- **Navbar.jsx** → Navigation bar used in all pages.

Context

- **AuthContext.jsx** → Manages user login and authentication.

API

- **api.js** → Handles API calls to backend.

Main Files

- **App.jsx** → Main app component.

- **main.jsx** → Entry point of the application.

- **App.css / index.css** → Styling files.

Assets

- **hero.png** → Image used in UI.

- **react.svg / vite.svg** → Icons used in project.

Config Files

- **vite.config.js** → Vite configuration.

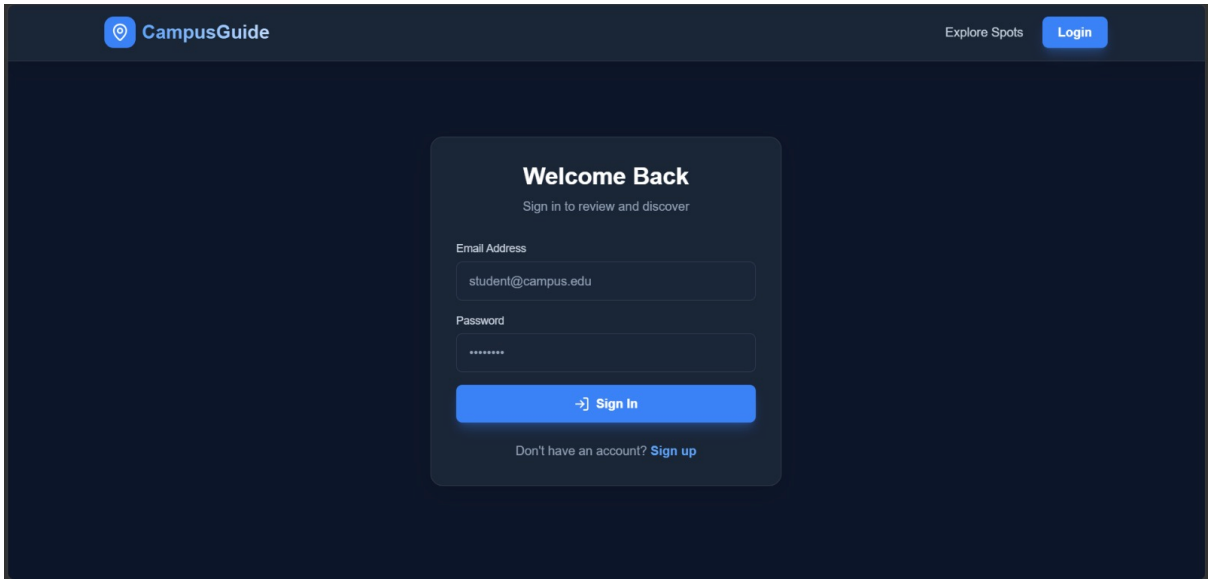
- **tailwind.config.js** → Tailwind CSS setup.

Public

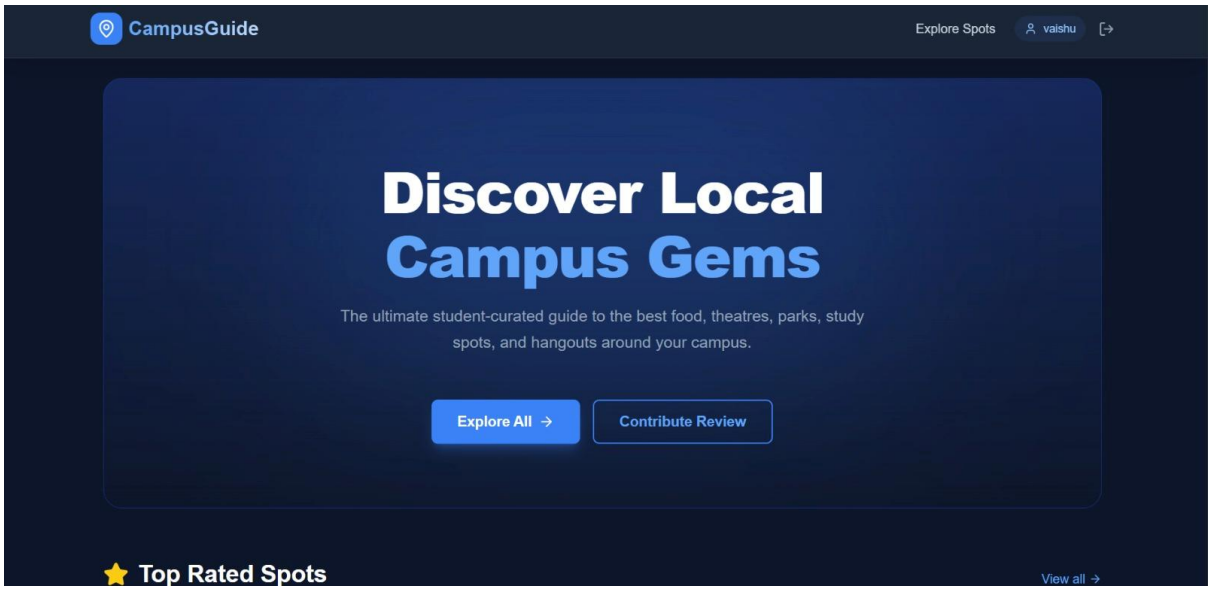
- **index.html** → Main HTML file.

Output screenshots:

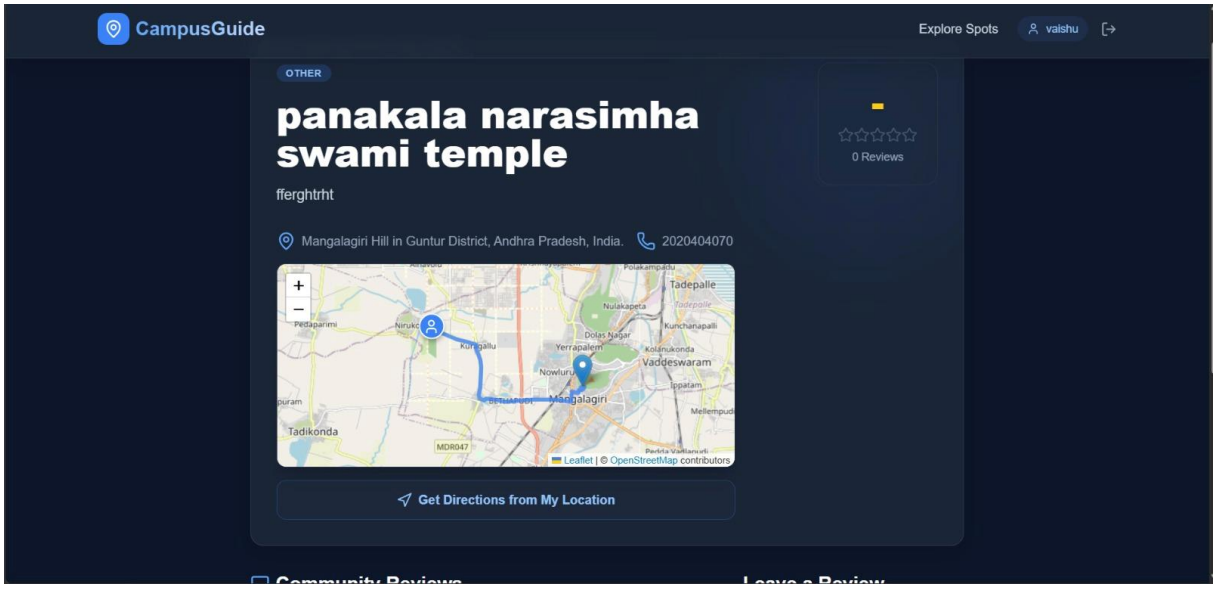
Login page:



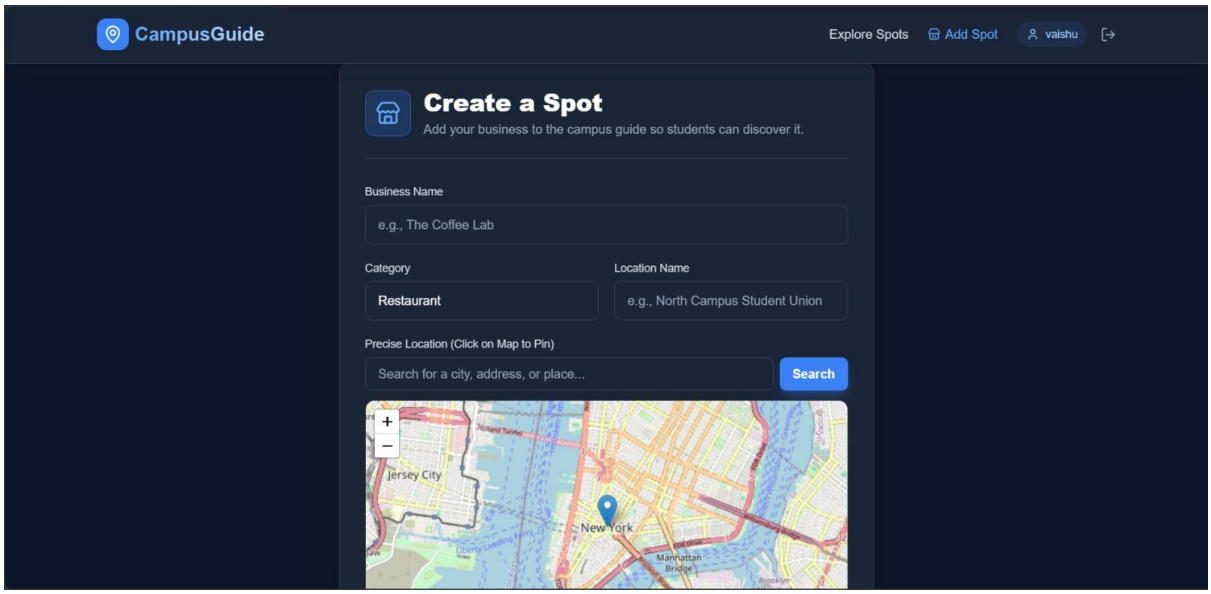
Home page:

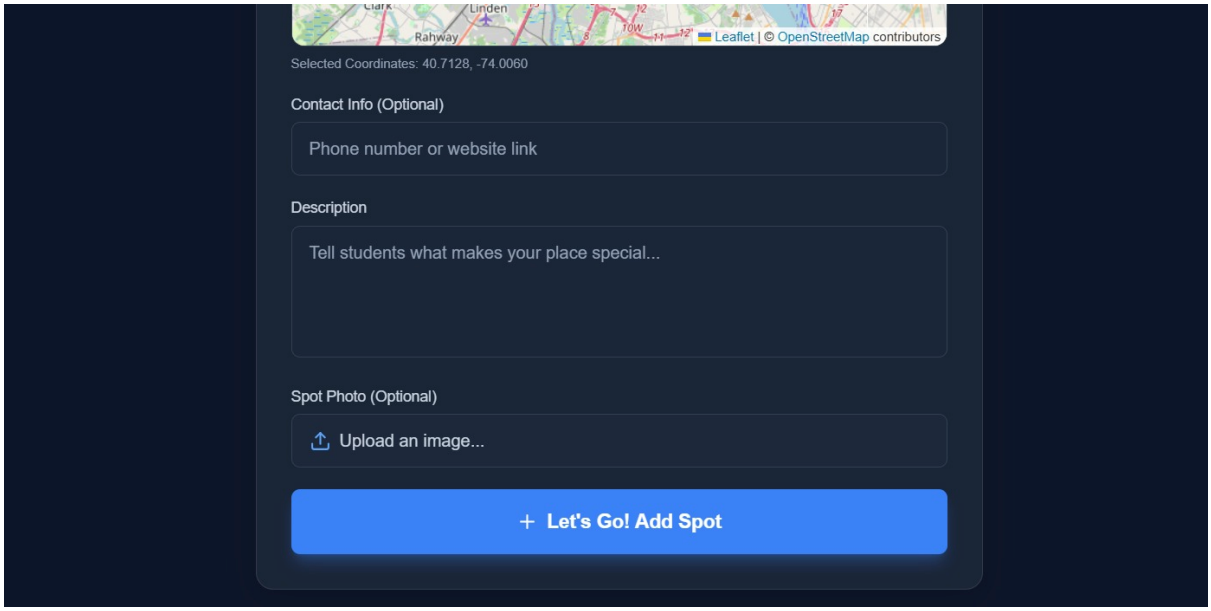


Exploring spots (user):



Adding spots:





Demo video link:

https://drive.google.com/file/d/1jyVdKIk_AwCAYK8oaujBhl2TydXsU2Ln/view?usp=drive_link

Code Drive Link:

https://drive.google.com/file/d/1_6Lx7Zb8KedqLtPqrA1AJfQdcxGQ7Dlf/view?usp=sharing